



NOTE: Using Claude AI or other LLM

- We expect that you will use it
- **I *LOVE* Claude Learner Mode!!! Opus 4.1**
- Prospective employees *expect* that you know how to use it
- Treat an LLM as a junior engineer who is helping you
 - You can ask Claude to do things for you, but...
 - It might (often) get it wrong
 - You are responsible for what it produces
 - You will have to understand what it produces because
 - You will have to debug what it produces
- TA's will “mock interview” every student **ONCE A WEEK**

Course Outline

- In this course you'll learn both theory and practical knowledge
 - To be able to engineer robust, scalable, efficient internet scale systems
- Includes basic material from
 - Networking (recommended course!)
 - CPU Hardware, processes and threads(Systems is also recommended!)
 - Virtual machines, containers
 - Web server basics
- Practical knowledge will be gained by
 - Labs using AWS Academy, Docker, Terraform
 - Assignments
 - Asking questions!

Readings!

- Papers and discussions
 - Piazza!
- Distributed Systems for Fun and Profit, Mikito Takada, ([online](#))
- Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems, Martin Kleppmann, O'Reilly 2017

Week 1 – Introduction to **Scalable** Systems

- Topics we will cover today

- *What* is Scalability?
- Basic Design Principles
- Tradeoffs!
- *How* do we Scale?!
- VMs
- Containers
- Threads?! Processes?

!

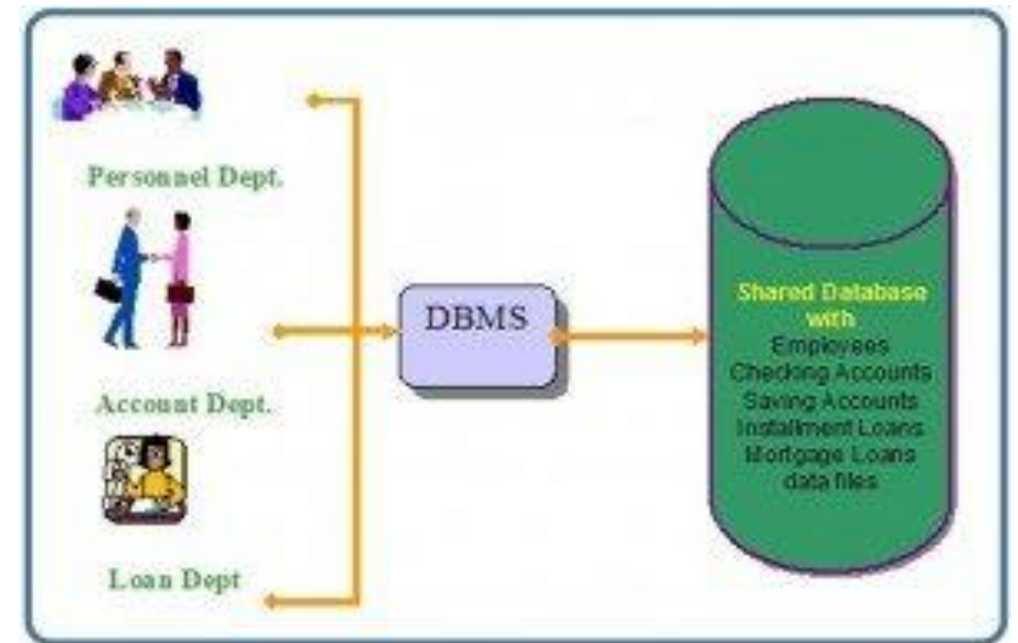
Examples of Scale, 2025 (2024)

- *16.4 (8.5) billion Google searches per day*
- *4.9 billion people use Google*
- *173 million ChatGPT searches per day*
 - *up from zero three years ago!*
- *www.demandsage.com and <https://explodingtopics.com/blog/google-searches-per-day>*

The same statistics published on websites that obviously scarped from each other. Unclear how accurate they are. An example of the challenges of doing research on the post-AI internet.

Impact of Scale on Business System Design

- Example: Bank System
 - Before Internet, easy to predict load on a system.
 - Post Internet, Imagine if the bank has 5 million customers.



What is Scalability

- Think of any physical system
 - Transit system
 - Airport
 - Elevators
- How we increase capacity is kind of *obvious*!?
- Linear response means
 - Add x% resources, get x% more scale
 - Never achievable in practice, there is always a bottleneck

Scaling Different Attributes of Software

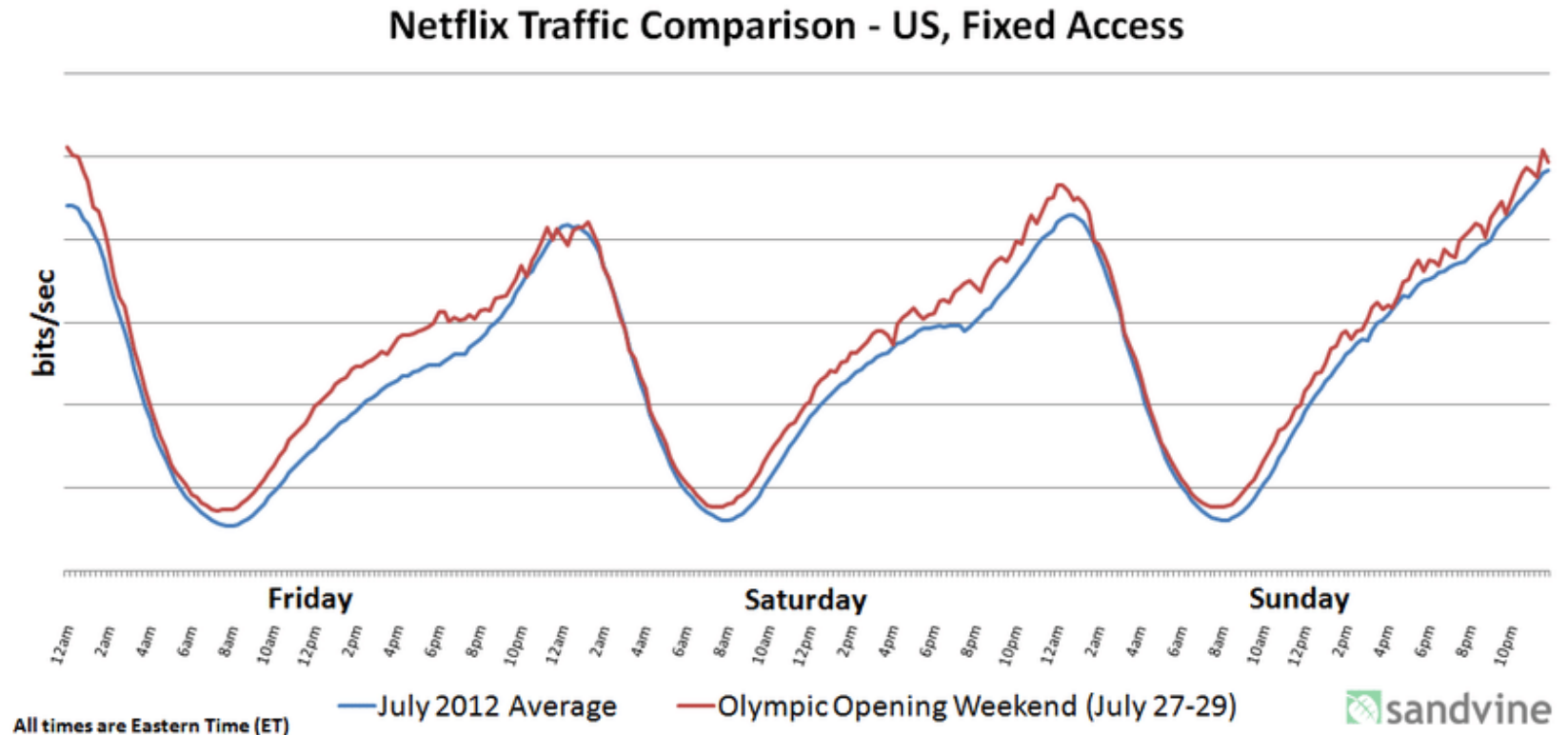
- Scalability for Software Systems is its capability to handle growth in some dimension, e.g.
 - The **number of simultaneous requests** a system can process
 - The **amount of data** a system can effectively process and manage
 - Maintain a stable, **consistent response time** as the request load grows

When to Scale?

- Early in system lifetime?
 - Scalability not a primary requirement
 - Emphasis on new features
- If system becomes successful...
 - More Users & More data
 - AND they use it in ways you didn't expect => more feature requests
- If a system prioritizes scaling early in a system lifetime, it can introduce additional complexity causing development inertia
- Design with an eye on future scaling
 - Do not make non-scalable decisions!!!

What is Scalability

- Key Terms
 - Scaling Up or Scaling Out – Increasing a Systems' capacity
 - Scaling Down – Reduce System Costs
- Example - Netflix



Scalability – **Basic** Design Principles

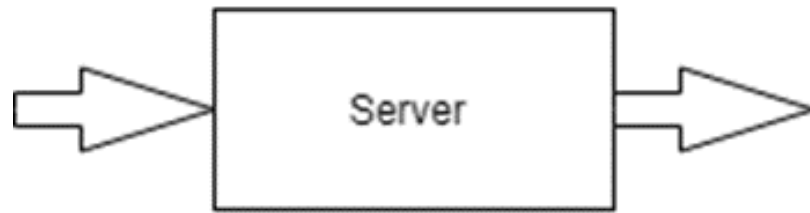
- Aim
 - increase capacity in some application specific dimension
- A common dimension
 - is increasing the number of requests that a system can process in a given time period also known as **throughput**.

Scaling-Up vs Scaling-Out

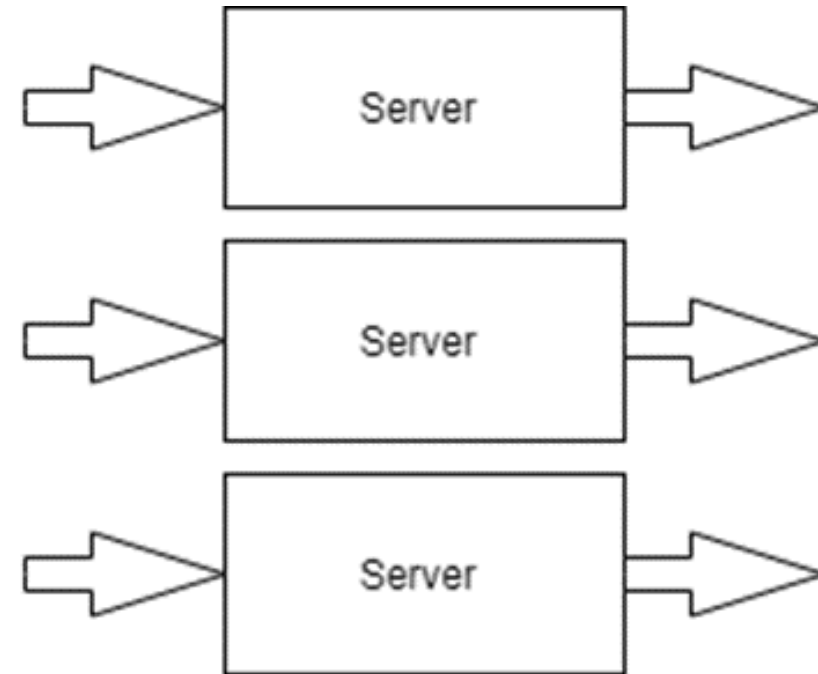
- Scaling Up (1980's solution)
 - Buy a bigger machine. Not cost-effective, not always possible
- Scaling Out (cloud solution)
 - Run copies of the software on a fleet of relatively inexpensive machines...

Increasing Capacity Using Replication

- Increasing Capacity - Replication



A single server can process 100 requests per second



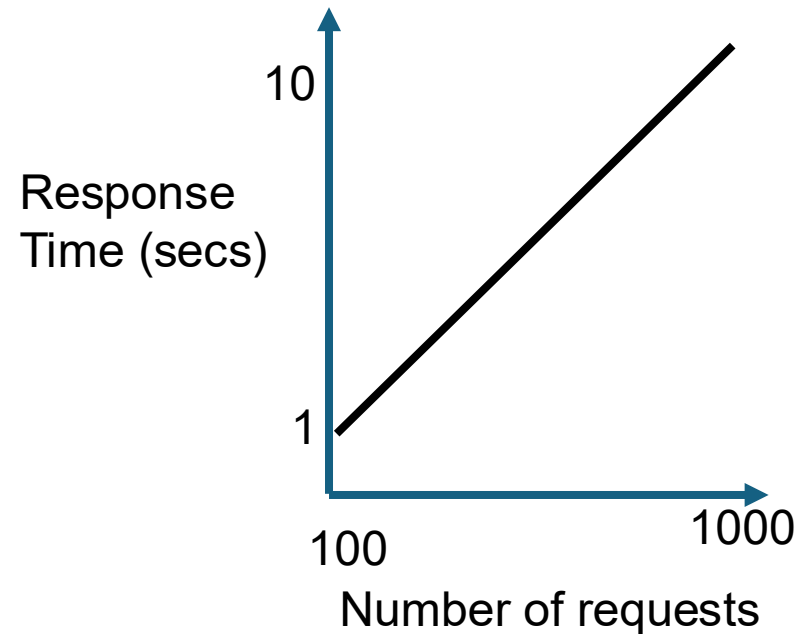
By replicating our server 3 times, we can process 300 requests per second

Write Better (More Efficient) Code

- **Optimize** processing to create more capacity
 - Using more efficient algorithms
 - Adding indexes to databases to speed up queries
 - Rewriting servers in a faster programming language
- **Example:** Facebook's creation of HipHop for PHP which increased speed of Facebook's web page generation by up to 6 times by compiling PHP code to C++

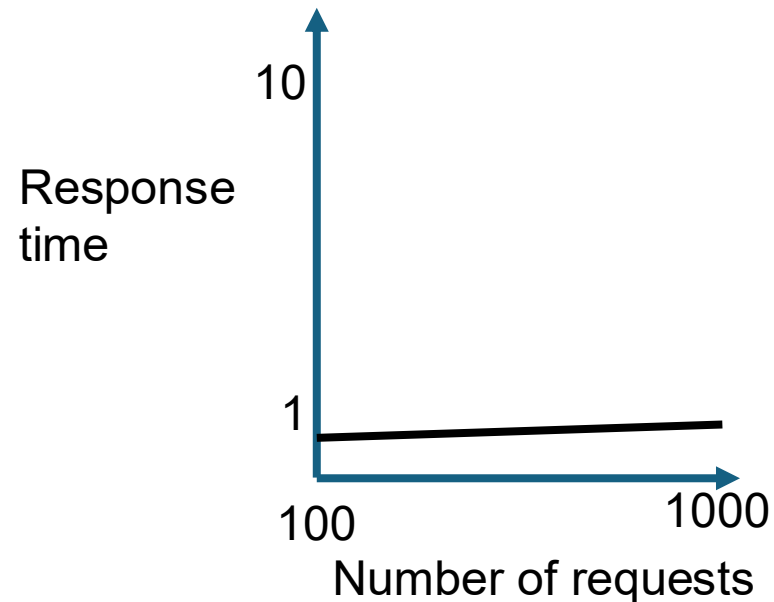
Cost to maintain acceptable Response Time

- Example – A Web based system
 - can service a load of 100 concurrent requests
 - mean response time of 1 second.
- Requirement to scale up this system to handle 1000 concurrent requests!
 - Without any changes, a simple load test reveals the below performance



Goal: Stable Response Time

- As the request load increases, we see the mean response time steadily grow to 10 seconds!!!!
- The system does **not scale with increasing traffic**
- After some engineering effort to improve performance, below is the graph!



What is Scalability

Adding one more instance of a server should always require the same small amount of effort

- Eg., doubling the transactions-per-second of a system should not cost double the price of the original server
 - Using **scaled-out mircoservices** it can be a very small fraction

BUT Scaling a system **not designed to scale** is REALLY hard!
EVEN WITH AI!!!!!!

What did the modification cost?

- Maybe simple and low cost?
 - More powerful web server
 - Multiple server replicas
- But what if it needed to be a
 - Rewrite of server code so that it can run on multiple servers
 - Optimize the server code for a single server
 - Database optimization
 - Rewrite server code using faster framework or language
 - Changes in hardware, or more hardware

Scalability And Costs

- Optimize server code = 100 development-hours
- Database schema and code changes = 500 development-hours
- Rewrite server code using fast framework or language = 10K hours
- A software system not designed to scale will cost more to transform
 - It's probably doable
 - But is it economical?
- Scalability = $f(\text{effort} * \text{cost}) + (\text{deployment} * \text{cost})$
- At scale, costs are significant!!!!
 - 10% reduction of \$10million per month is significant

Scalability Goal

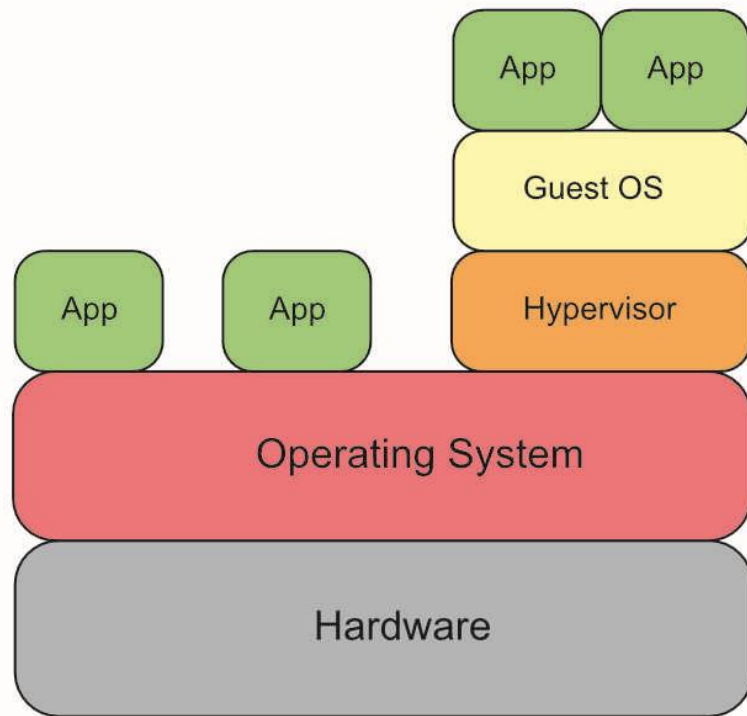
- Achieve the required capacity at lowest cost when scaling a system!
- Software systems that can be **scaled exponentially** while costs grow linearly are known as **hyperscale systems!!!**

HOW DO WE SCALE?!?!?!?
Virtual Machines and Containers!

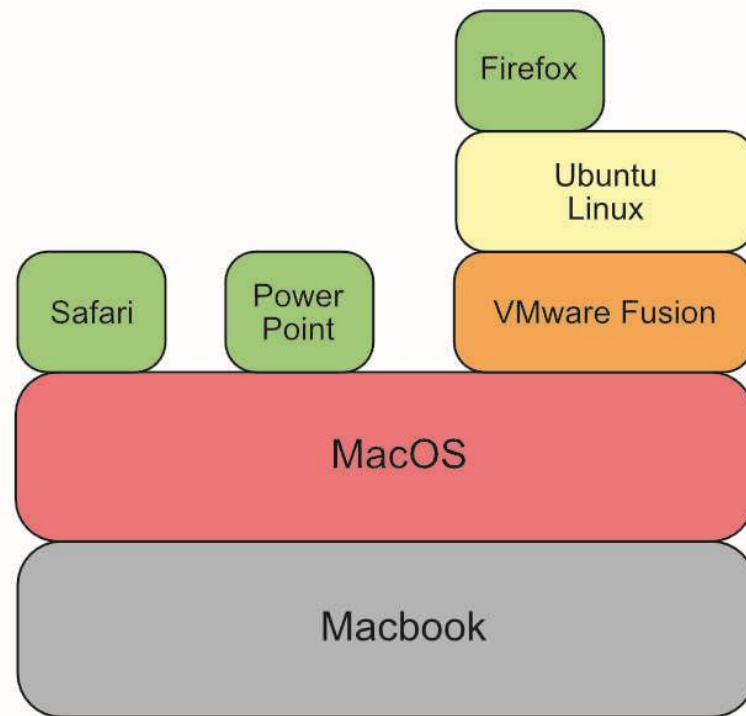
Virtual Machines (VMs)

- Informally: Run several different “computers” on the same hardware
- Guest Operating Systems be different operating systems
- The basic unit of **compute** available from the cloud
 - AWS EC2
 - Google Compute Engine
 - Azure Virtual Machines
- Azure has a Microsoft Windows substrate, but you can rent Linux VMs!!!
- Completely separate running operating system, cannot access one OS from the other

Virtual Machine Layers

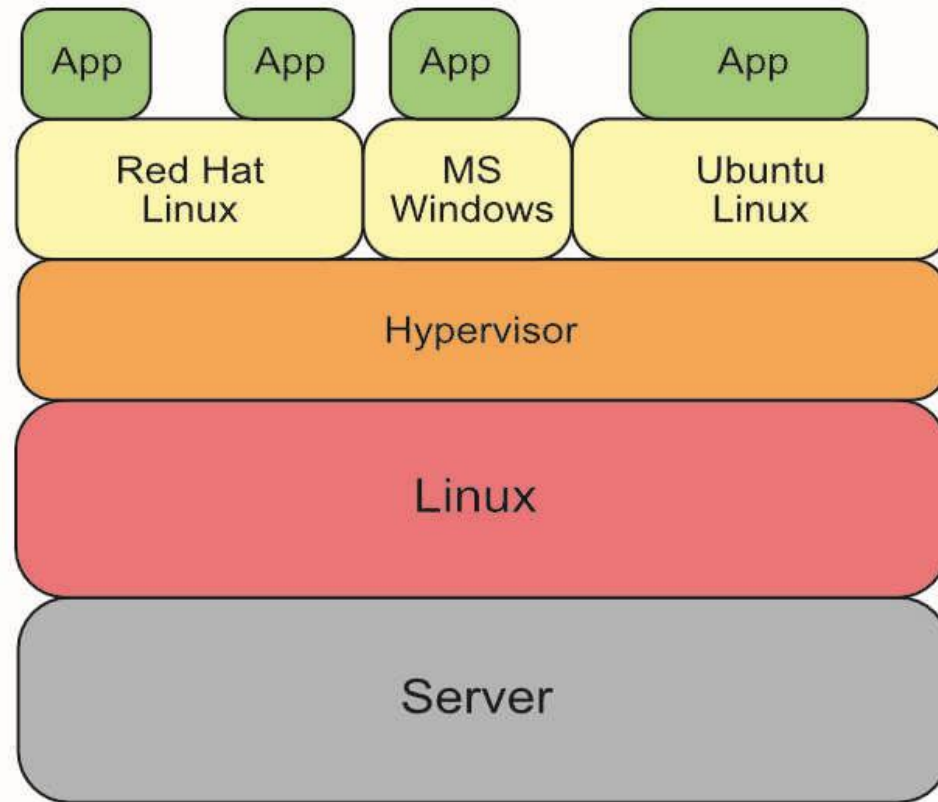


Layers



Example

Virtual Machine “Farm” in the Cloud



VM "Farm"

Pros and Cons of VMs

- Pros

- Can **share hardware** between separate users
- High Security
- Easy to create and destroy
- VM description can be stored!

- Cons

- Additional complexity...
- Very slight reduction in performance, perhaps 2%

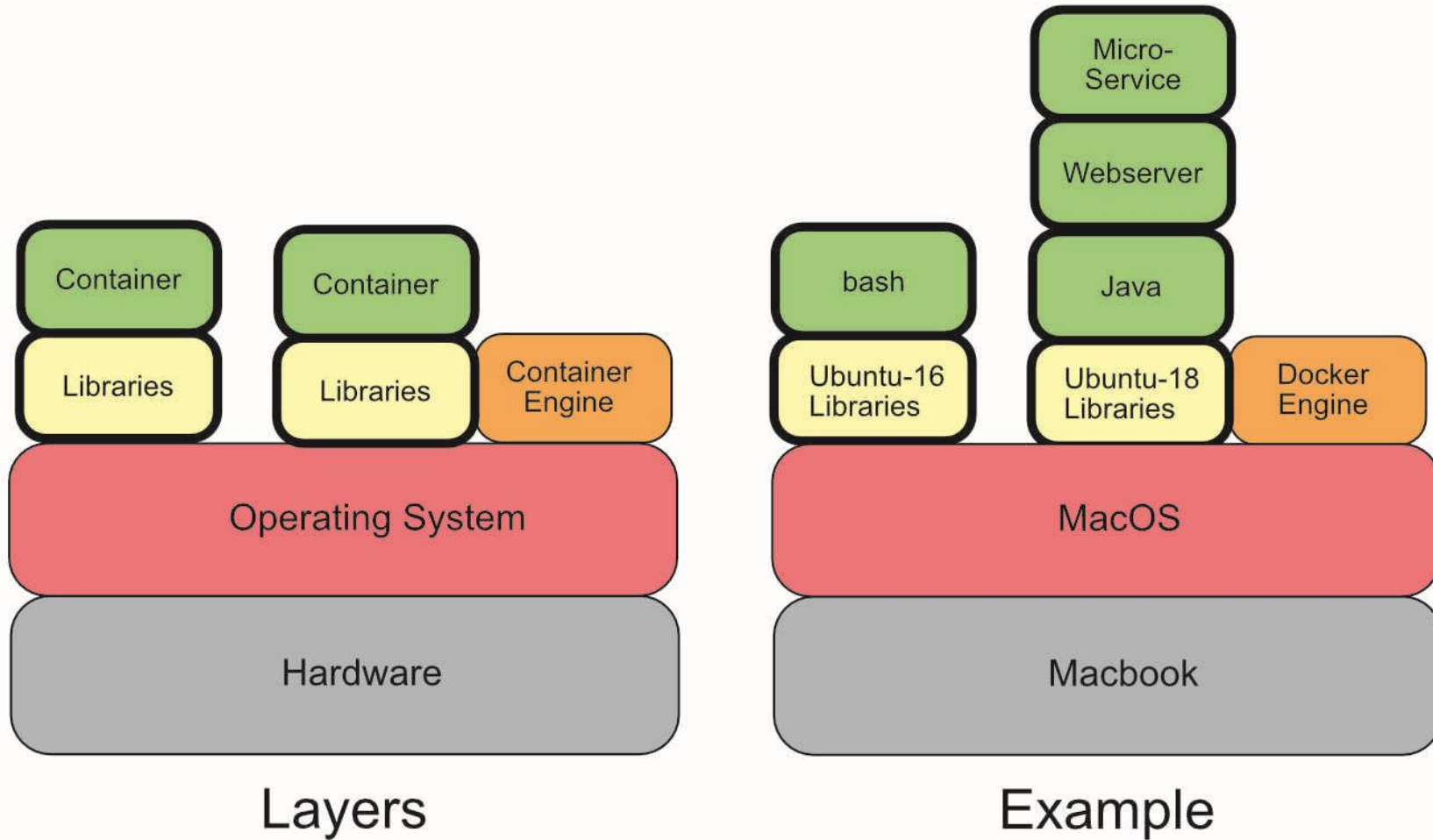
Docker & Containers

- Docker is a popular system for running lightweight containers
- A container is just a set of processes within an OS, but...
 - Process within the container-group cannot affect other non-container processes!!!
 - Processes cannot “cd” out of a defined directory (*chroot*)

CONCEPTS!

- Thread is to Process and Container is to VM
- Containers and Threads are lightweight with a little less security

Container Layers



Dockerfiles

- Extends an existing OS build by adding
 - Software!!! It contains
 - Libraries
 - Your Code
 - Static Data
 - What it runs when it starts...
 - Best Practice, a container is built for *exactly one application*
 - The language looks like bash, but it isn't! Be careful!!!!
- **Docker Hub** contains thousands of base images
 - Plain Operating Systems (e.g. Ubuntu)
 - Operating Systems plus a language (e.g. Ubuntu plus Python)

Use a base image with OpenJDK

FROM openjdk:17-jdk-slim

Set the working directory inside the container

WORKDIR /app

Copy the application's JAR file into the container

Assuming your JAR file is named 'your-application.jar' and is in the same directory as the Dockerfile

COPY your-application.jar /app/app.jar

Expose the port your application listens on (if it's a web application)

EXPOSE 8080

Define the command to run the application when the container starts

ENTRYPOINT ["java", "-jar", "app.jar"]

Use an official Python runtime as a parent image

FROM python:3.9-slim-buster

Set the working directory in the container

WORKDIR /app

Copy the current directory contents into the container at /app

COPY . /app

Install any needed packages specified in requirements.txt

RUN pip install --no-cache-dir -r requirements.txt

Make port 8000 available to the world outside this container

EXPOSE 8000

Define environment variables

ENV NAME World

Run app.py when the container launches

CMD ["python", "app.py"]

Technologies

- MacOS
 - VMWare Fusion – Virtual Machines
 - Docker Desktop – Containers
- AWS
 - **EC2** – Virtual Machines
 - **ECR** Elastic Container Registry – Container Images
 - **ECS** Elastic Container Service – Running images as containers

Summary – Containers

- Operating System Support (Linux, MacOS, Unix)!
 - **chroot** – processes cannot change directory out of their own
 - **Process groups** – the processes within the container are controlled as one
- Pros and cons
 - Faster to spin up and down than VMs
 - Less secure than VMs
 - Slightly more efficient

Questions?
HIT PIAZZA!!!!!!!!!!!!
Claude Learner Mode!